

The Ten-Line CTA: What Five Structural Refinements Actually Buy

Thomas Schmelzer*

October 6, 2026

Abstract

This repository contains a trend-following strategy written in ten lines of code and four successive refinements of it, each a marimo notebook whose sliders expose its free parameters. This paper measures what the sequence buys. The headline Sharpe ratio rises from 0.56 to 1.47 over the five systems on the shipped panel of 49 futures and 53 years of daily data, but it does not rise monotonically: the third step, which introduces the Huber clip and the analytically scaled oscillator, leaves the Sharpe ratio unchanged to three decimals ($0.8794 \rightarrow 0.8776$) and would be read as a null result by anyone looking only at that number. It is not a null result. It cuts annual one-way turnover by a factor of three, and net of a one-way cost of 5 basis points it is worth +0.14 Sharpe over the step it replaces. Turnover is in fact where the sequence is least flattering to itself: it rises 35-fold from the first system to the last, so the ranking of the five is cost-dependent, and beyond roughly 12 basis points the ten-line original beats the correlation-optimised finale. Against that, we find the structural steps robustly dominate parameter search. Sweeping the two moving-average lengths over all 852 admissible pairs, the *best* pair for any system is worse than the *default* parameters of the system two steps later, and the Spearman rank correlation between the parameter surfaces of any two systems is between 0.93 and 0.98 — the structural change moves the level, not the shape. Optuna searches over the same spaces add +0.06 to +0.23 Sharpe, against +0.32, +0.19 and +0.39 for the three structural steps that pay.

1 Introduction

The premise of this repository is a challenge: write a hedge-fund strategy in ten lines of code. The answer given is a sign-of-a-crossover trend follower, a construction old enough that its literature is about how long it has worked (Hurst et al., 2017) rather than whether it works (Moskowitz et al., 2012). It is also, in the form first written, indefensible in several specific ways that the notebook itself lists: no notion of risk, a bet size independent of the asset, and a signal whose only values are ± 1 .

What follows in the repository is not a defence of the ten lines but four rewrites, each addressing one flaw and each still small enough to read in one screen. The sequence is deliberately pedagogical: volatility scaling, then a price filter and a scale-free oscillator, then cross-sectional risk scaling, then a full covariance-aware allocation. That last step is convex optimisation in the sense of Schmelzer and Hauser (2013): the position is the solution of a small linear system involving an estimated correlation matrix, and the parameter-estimation care that paper argues for is what makes it usable.

This paper asks the question the notebooks leave open. Each step *improves the Sharpe ratio* — that is what the printed number at the bottom of each notebook shows, and it is what the sequence is arranged to demonstrate. But a Sharpe ratio computed gross of costs on a

*<https://github.com/tschm/cs>

single panel is a weak instrument, and three questions are more interesting than the ranking it produces:

- What does each step cost in turnover, and does the ranking survive trading costs?
- How much of the improvement could have been had by tuning the parameters of the previous step instead — the “parameter-hacking” the notebooks explicitly disclaim?
- Which of the five steps are doing something the headline number does not show?

Section 2 states the five systems as formulas. Section 3 records the implementation choices that are not implied by those formulas and matter for reproducing the numbers. Section 4 answers the three questions. The short version of the answers: costs reorder the five and can invert the sequence entirely; parameter search is worth roughly half a structural step and never more; and the step that looks worthless is the one that makes the rest affordable.

2 The five systems

Let $p_{t,i}$ be the price of asset i at time t , and let ewm_m denote an exponentially weighted mean with centre of mass m (so decay $1 - 1/(m + 1)$ per step), with ewm_m^σ the matching standard deviation. Every system maps prices to a *cash position* $x_{t,i}$, which is then run through the portfolio accounting of `jquantstats` against an AUM of 10^8 . Long-only is never imposed and no leverage limit is applied; the scaling constant in each system is chosen so that the resulting portfolio volatility is of a comparable order.

CTA 1.0 — the ten lines.

$$x_{t,i} = 5 \cdot 10^6 \cdot \text{sign}(\text{ewm}_{32}(p_{\cdot,i})_t - \text{ewm}_{96}(p_{\cdot,i})_t). \quad (1)$$

Each asset carries $\pm 5\%$ of AUM in notional, whatever it is. The position is piecewise constant and changes only when the crossover flips sign.

CTA 2.0 — volatility scaling.

$$x_{t,i} = 10^5 \cdot \frac{\text{sign}(\text{ewm}_{32}(p_{\cdot,i})_t - \text{ewm}_{96}(p_{\cdot,i})_t)}{\text{ewm}_{32}^\sigma(\Delta p/p)_{t,i}}. \quad (2)$$

The bet is now inversely proportional to the asset’s estimated volatility, so a bond future and an equity future are sized to contribute comparably. This is the step that turns a signal into a portfolio. Note what it does to trading: the numerator still only flips, but the denominator moves every day, so the position moves every day.

CTA 3.0 — filter the price, scale the signal. Equation (2) feeds a raw price into a function that is only scale-free because sign discards magnitude. To use anything richer than a sign, the price has to be standardised first. Define clipped, volatility-adjusted log returns

$$\tilde{r}_{t,i} = \text{clip}_c \left(\frac{\Delta \log p_{t,i}}{\text{ewm}_{\nu-1}^\sigma(\Delta \log p_{\cdot,i})_t} \right), \quad \text{clip}_c(z) = \max(-c, \min(c, z)), \quad (3)$$

with $c = 4.2$ by default, and integrate them into an adjusted price $\tilde{p}_{t,i} = \sum_{s \leq t} \tilde{r}_{s,i}$. The clip is the derivative of the Huber loss (Huber, 1964) applied to the standardised score, which is what makes “we apply Huber functions” in the notebook an accurate description of an operation that looks like plain winsorising.

On the standardised price, the oscillator

$$\text{osc}(\tilde{p}; f, s)_t = \frac{\text{ewm}_{f-1}(\tilde{p})_t - \text{ewm}_{s-1}(\tilde{p})_t}{\varsigma(f, s)}, \quad \varsigma(f, s)^2 = \frac{1}{1 - \phi^2} - \frac{2}{1 - \phi\gamma} + \frac{1}{1 - \gamma^2}, \quad (4)$$

with $\phi = 1 - 1/f$ and $\gamma = 1 - 1/s$, divides the crossover by its theoretical standard deviation under a unit-variance random walk. The point of ς is that the oscillator has unit scale for *any* pair (f, s) , so no separate volatility lookback is needed to compare a fast crossover with a slow one. The system is then

$$x_{t,i} = 10^5 \cdot \frac{\tanh(\text{osc}(\tilde{p}_{\cdot,i}; f, s)_t)}{\text{ewm}_s^\sigma(\Delta p/p)_{t,i}}. \quad (5)$$

The $\tanh$ bounds the signal, so a large oscillator reading no longer translates into an unbounded position.

CTA 4.0 — cross-sectional risk scaling. Write $\mu_t \in \mathbb{R}^N$ for the vector of signals $\mu_{t,i} = \tanh(\text{osc}(\tilde{p}_{\cdot,i}; f, s)_t)$. The first three systems treat the assets one at a time; this one normalises across them,

$$x_{t,i} = 5 \cdot 10^5 \cdot \frac{1}{\text{ewm}_\nu^\sigma(\Delta p/p)_{t,i}} \cdot \frac{\mu_{t,i}}{\|\mu_t\|_2}, \quad (6)$$

so the total signal strength spent on any day is constant: conviction is redistributed between assets rather than added to the book. This is the smallest possible step from a univariate to a portfolio construction — it uses the cross-section but not the covariance.

CTA 5.0 — correlation-aware allocation. The last system replaces the Euclidean norm in (6) by one induced by an estimated correlation matrix. Per-day correlations come from an exponentially weighted covariance of the standardised returns $\tilde{r}$ in the manner of Engle (2002),

$$\Sigma_{t,ij} = \text{ewm}_\kappa(\tilde{r}_i \tilde{r}_j)_t - \text{ewm}_\kappa(\tilde{r}_i)_t \text{ewm}_\kappa(\tilde{r}_j)_t, \quad C_{t,ij} = \frac{\Sigma_{t,ij}}{\sqrt{\Sigma_{t,ii} \Sigma_{t,jj}}}, \quad (7)$$

shrunk towards the identity (Ledoit and Wolf, 2004) with intensity λ ,

$$\tilde{C}_t = (1 - \lambda)C_t + \lambda I. \quad (8)$$

Restricting to the assets with a live price on day t , the position is

$$x_t = 10^6 \cdot D_t^{-1} \frac{\tilde{C}_t^{-1} \mu_t}{\sqrt{\mu_t^\top \tilde{C}_t^{-1} \mu_t}}, \quad D_t = \text{diag}(\text{ewm}_\nu^\sigma(\Delta p/p)_{t,\cdot}). \quad (9)$$

The numerator is the direction that maximises signal per unit of estimated risk; the denominator normalises that risk to one. Days on which the solve fails, or the norm vanishes, are left flat rather than allowed to raise an exception out of the loop.

Equation (9) is the only system that inverts anything, and it is the reason the shrinkage parameter exists: the inversion is the step Michaud (1989) warns about and Schmelzer and Hauser (2013) treat as a numerical-hygiene problem. Nothing here estimates expected returns, so the comparison with mean–variance optimisation (Markowitz, 1952) is only partial — μ_t is a bounded signal, not a return forecast.

3 Implementation

The notebooks are the source of truth. Each system lives in one marimo notebook (Agrawal et al., 2024) that defines a signal function $\mathbf{f}$ and builds a portfolio from it. The Optuna driver (Akiba et al., 2019) does not reimplement the strategies: it executes each notebook with `runpy` and pulls $\mathbf{f}$ out of the resulting namespace, so a parameter search cannot silently drift away from the strategy it claims to be searching. The five headline Sharpe ratios are pinned in the test suite to a relative tolerance of 10^{-6} and checked from both directions — once by executing the notebooks, once by calling the driver’s builders — so the numbers in Section 4 are regression-tested, not just recorded.

Warm-up. The EWM estimators are given `min_samples` of 100 (CTA 1.0) and 300 (all later systems). With 13,909 daily rows the warm-up is immaterial to the reported statistics, but it is the reason a null-safe fill is applied before the portfolio is built: the first rows of every signal are undefined, and they are set to zero, meaning flat, not dropped.

Two parameters that do not do what their labels say. Two details are worth recording because they are invisible in the formulas and visible in the results. In CTA 3.0, the volatility used to *size* the position in (5) has centre of mass s , the slow crossover length — not ν , the value the “Volatility” slider sets. That slider reaches only the price filter (3). So in CTA 3.0 the slow parameter does double duty, and the shape of its parameter surface is not comparable with that of its neighbours for that reason. In CTA 5.0 the crossover lengths are fixed at (32, 96) in the driver and are not search dimensions at all; what is searched there is (ν, κ, λ) .

Numerical conventions. The panel is read from a CSV of daily prices with hashed column names, interpolated across internal gaps, and cast to `Float64`. Returns are simple returns for the volatility estimates in (2), (5), (6) and (9), and log returns for the price filter (3). Covariance and correlation are the plain exponentially weighted sample estimators; the only regularisation anywhere is the shrinkage (8). Signals are computed with Polars (Vink et al., 2024) expressions and the per-day solve of (9) with NumPy (Harris et al., 2020).

4 Results

All numbers come from the panel shipped with the repository: 13,909 daily rows from 1970-01-02 to 2023-04-26, 49 assets with hashed identifiers, between 3,556 and 13,908 observations each (median 9,467). The annualisation factor implied by the date index is 260.70 periods per year. Sharpe ratios are gross of costs and computed on the portfolio’s daily returns unless stated otherwise; “turnover” is annual one-way turnover as a multiple of AUM, that is, the sum of absolute position changes over all assets and days divided by AUM and by the number of years.

4.1 The headline sequence

Table 1 is the sequence as the notebooks present it, plus the columns they do not print. Three readings of it are worth separating.

The Sharpe ratio rises by a factor of 2.6 end to end, but in four steps of which one is not a step: +0.319, −0.002, +0.194, +0.394. CTA 3.0 is flat against CTA 2.0 — 0.8776 against 0.8794 — and marginally worse. Anyone reading the printed Sharpe ratio at the bottom of each notebook sees the third refinement do nothing.

The risk columns tell a cleaner story than the return column, and they are monotone where the Sharpe ratio is not. Maximum drawdown improves at every single step, from −61% to −35%. Excess kurtosis falls from 30.5 to 1.4: CTA 1.0’s daily return distribution is dominated

Table 1: The five systems at their notebook default parameters. Sharpe ratios are gross of costs. “Params” counts the sliders the notebook exposes.

System	Sharpe	Ann. vol.	Max DD	Kurtosis	Skew	Turnover	Params
CTA 1.0	0.5606	14.41%	−61.44%	30.54	+0.49	4.2	2
CTA 2.0	0.8794	18.10%	−55.47%	6.44	−0.41	86.8	3
CTA 3.0	0.8776	13.89%	−45.55%	7.99	−0.41	27.7	4
CTA 4.0	1.0712	15.51%	−38.82%	3.94	−0.36	64.3	4
CTA 5.0	1.4652	18.58%	−35.30%	1.44	−0.17	147.5	6

Table 2: Sharpe ratio net of a linear one-way trading cost, charged as $\text{cost}_t = \text{turnover}_t \times \text{bps} / 10^4$ on the daily portfolio return.

System	0 bps	1 bps	2 bps	5 bps	10 bps	20 bps
CTA 1.0	0.5606	0.5576	0.5547	0.5459	0.5312	0.5018
CTA 2.0	0.8794	0.8313	0.7832	0.6387	0.3981	−0.0776
CTA 3.0	0.8776	0.8577	0.8378	0.7779	0.6780	0.4780
CTA 4.0	1.0712	1.0297	0.9881	0.8632	0.6551	0.2404
CTA 5.0	1.4652	1.3849	1.3045	1.0633	0.6631	−0.1174

by rare enormous days — the inevitable consequence of a ± 1 signal on 49 assets sized without reference to their volatility — and by CTA 5.0 the distribution is close to Gaussian. Skewness moves from +0.49 to −0.17, which is the familiar trade: the crude system is a lottery ticket, the refined one is an insurance seller. Both facts matter for how the Sharpe ratio should be read at all, since its estimation error grows with both (Lo, 2002): the same 0.56 measured on a kurtosis-30 series and on a kurtosis-1.4 series are not equally trustworthy numbers.

And turnover rises by a factor of 35. This is the column that reorders the table, and it is the subject of the next section.

4.2 Costs reorder the sequence

Turnover in Table 1 is not monotone, and its pattern is the inverse of what the notebooks suggest. CTA 1.0’s own commentary calls the sign function “very expensive to trade as position changes are too extreme”, which is true of each individual trade and false of the total: the sign function is piecewise constant, so it trades only on a crossover flip, and its 4.2 turnovers a year make it by a wide margin the *cheapest* of the five. Volatility scaling is what makes trading continuous — the denominator of (2) moves daily — and it multiplies turnover by twenty, to 86.8.

Table 2 charges a linear cost against turnover and recomputes the Sharpe ratio. The step that looked like nothing is the only step in the sequence that *reduces* turnover, from 86.8 to 27.7, and the reason is in (5): bounding the signal with tanh and standardising the price with (3) removes exactly the large signal excursions that generate large position changes. Net of 5 basis points, CTA 3.0 beats CTA 2.0 by +0.139 Sharpe; net of 10, by +0.280. The break-even is at 0.06 basis points — effectively, CTA 3.0 dominates CTA 2.0 at any cost above zero. The refinement is worth what it is worth entirely in trading costs, and the headline number cannot see it.

Table 3 generalises this into the full ordering. Two entries deserve attention. Above 7.05 basis points, the ten-line original beats volatility scaling; above 11.72, it beats the correlation-optimised finale, and above 18.60 it beats every other system in the sequence. And beyond roughly 9 to 10 basis points, CTA 3.0 — the step that appeared to contribute nothing — beats

Table 3: One-way cost, in basis points of traded notional, at which the net Sharpe ratios of the row and column systems coincide. “—” on an off-diagonal entry means no crossing below 40 bps: CTA 4.0 dominates CTA 2.0 at every cost level.

	CTA 1.0	CTA 2.0	CTA 3.0	CTA 4.0	CTA 5.0
CTA 1.0	—	7.05	18.60	13.20	11.72
CTA 2.0	7.05	—	0.06	—	18.64
CTA 3.0	18.60	0.06	—	8.94	9.75
CTA 4.0	13.20	—	8.94	—	10.21
CTA 5.0	11.72	18.64	9.75	10.21	—

Table 4: Sharpe ratio over the full grid of 852 admissible crossover pairs (f, s) , $f \in \{4, 8, \dots, 96\}$, $s \in \{f + 4, \dots, 192\}$, with all other parameters at their defaults. The last column is the percentile of the notebook default (32, 96) within that grid.

System	Min	q_{25}	Median	q_{75}	Max	SD	arg max	Default pct.
CTA 1.0	0.3035	0.4330	0.4892	0.5648	0.6433	0.0774	(4, 64)	72.5%
CTA 2.0	0.6030	0.7453	0.8183	0.8816	1.0016	0.0857	(4, 84)	73.6%
CTA 3.0	0.6356	0.7706	0.8333	0.8859	0.9402	0.0717	(4, 76)	70.7%
CTA 4.0	0.7759	0.9183	1.0104	1.0927	1.2260	0.1059	(4, 76)	67.4%

both of the optimised systems that follow it.

How much this matters depends on the cost of the panel, which the anonymised data does not let us estimate. For liquid futures a one-way cost of one to three basis points of notional is a defensible order of magnitude, and in that range the gross ranking survives: at 2 bps the five systems are still ordered $0.55 < 0.78 \approx 0.84 < 0.99 < 1.30$. The honest summary is not that the sequence is wrong but that its margin is thinner than the gross numbers suggest, that it is monotone in cost-adjusted terms only below about 9 bps, and that a strategy trading 147 times its AUM a year is a different kind of object from one trading 4 times, whatever their Sharpe ratios.

4.3 Structure against parameters

The notebooks state a thesis about parameter tuning: that “such fundamental flaws are not addressed by parameter-hacking or pimp-my-trading-system steps”. This is testable, and it holds — with a margin large enough to be worth quantifying.

Table 4 sweeps the two parameters every practitioner tunes over their entire admissible grid. CTA 5.0 is absent because its crossover lengths are fixed in the driver and are not a search dimension there.

The central comparison is between the maximum of one row and the default of a later one. CTA 1.0’s best pair over all 852 — 0.6433 at (4, 64) — is below CTA 2.0’s median (0.8183), let alone its default (0.8794). CTA 2.0’s best (1.0016) is below CTA 4.0’s median (1.0104) and default (1.0712). CTA 4.0’s best (1.2260) is below CTA 5.0’s default (1.4652). Exhaustively tuning the two parameters of any system in this sequence buys less than adopting the structure of the system two steps later, and in the first case buys less than adopting the structure one step later.

Two further readings. First, the hand-set defaults are good but not tuned: (32, 96) sits between the 67th and 74th percentile of each grid, which is roughly where a sensible prior lands and well short of the maximum — consistent with sliders set by judgement rather than fitted. Second, every grid is maximised at the fastest admissible $f = 4$, which on daily data

Table 5: Spearman rank correlation of the Sharpe ratio between systems, over the 852 shared grid points.

	CTA 2.0	CTA 3.0	CTA 4.0
CTA 1.0	+0.978	+0.961	+0.966
CTA 2.0		+0.925	+0.951
CTA 3.0			+0.963

Table 6: Optuna TPE search (seed 42) over each system’s own space, with the repository’s default trial budgets. The final column is the structural gain from the preceding system at default parameters, for comparison.

System	Trials	Baseline	Best	Best parameters	Structural step
CTA 1.0	200	0.5606	0.6208	$f=20, s=52$	—
CTA 2.0	200	0.8794	1.0263	$f=4, s=84, \nu=16$	+0.319
CTA 3.0	150	0.8776	1.0000	$f=48, s=56, \nu=4$	−0.002
CTA 4.0	150	1.0712	1.3042	$f=4, s=80, \nu=12$	+0.194
CTA 5.0	40	1.4652	1.5936	$\nu=12, \kappa=280, \lambda=0.8$	+0.394

is a four-day centre of mass. That is a warning rather than a recommendation: the fastest crossover is also the most expensive to trade, and Table 4 is gross of costs.

Table 5 explains why. The parameter surfaces of the four systems are almost the same surface: the rank correlation between any pair is between 0.925 and 0.978. Whatever the structural refinements are doing, they are not changing which crossover lengths work. They shift the level of the whole surface and leave its shape essentially intact. This is the precise sense in which structure and parameters are not substitutes: a parameter search moves you along a ridge, and each structural step lifts the entire ridge.

Table 6 repeats the exercise with the repository’s own Optuna driver, searching the full space of each system rather than two of its parameters. The searches find +0.06 to +0.23 Sharpe. The three structural steps that pay find +0.32, +0.19 and +0.39. Only for CTA 4.0 does the search exceed the structural step that produced the system it is searching, and even there its best (1.3042) remains below CTA 5.0’s untuned default.

4.4 How much of the search gain is selection?

A best-of- K Sharpe ratio is biased upwards, and the size of the bias is the question Bailey and López de Prado (2014) and Harvey and Liu (2015) exist to answer. The standard correction subtracts the expected maximum of K draws,

$$\mathbb{E}[\max_K] \approx \sigma \left[(1 - \gamma_e) \Phi^{-1} \left(1 - \frac{1}{K} \right) + \gamma_e \Phi^{-1} \left(1 - \frac{1}{Ke} \right) \right], \quad (10)$$

with γ_e the Euler–Mascheroni constant, which for $K = 852$ gives a factor of 3.209 on σ . Which σ is the whole difficulty.

Taking σ as the cross-sectional standard deviation of the grid Sharpe ratios, as Bailey and López de Prado (2014) prescribe, gives haircuts of 0.248, 0.275, 0.230 and 0.340 for the four systems. Every one of these exceeds the entire observed distance from the grid median to the grid maximum — 0.154 for CTA 1.0, 0.183 for CTA 2.0. Read literally, the correction says the best point of each grid is *below* what pure selection would produce from identical strategies, which is not a coherent conclusion. It is the expected one: the correction assumes K independent trials, and Table 5 has just shown that neighbouring grid points are near

duplicates of each other. The effective number of independent trials in a 852-point grid over two smoothing lengths is far smaller than 852, and nothing here estimates it.

Taking σ instead as the sampling standard deviation of a single Sharpe ratio, $\sqrt{(1 + \text{SR}^2/2)/T}$ with $T = 13,909$ (Lo, 2002), gives haircuts of 0.029 to 0.034. That is a lower bound in the opposite direction, since it ignores the parameter search entirely.

The useful statement is the bracket. The selection bias in Table 6 is somewhere between 0.03 and 0.34 Sharpe, and the structural steps of +0.32, +0.19 and +0.39 are large relative to the lower end and comparable to the upper end. This does not overturn Section 4’s conclusion — the structural steps are measured at fixed default parameters, so they carry no selection bias of their own — but it does mean the search gains in Table 6 should be read as upper bounds on what a search would deliver out of sample, and quite possibly as nothing at all.

5 Limitations

Everything above is in-sample in the strongest sense: one panel, one realisation, all statistics computed on the same data that produced the signals, and no walk-forward anywhere. The Sharpe ratios in Table 1 are properties of a construction applied to a history, not estimates of what any of these systems would earn. The panel runs from 1970, and a trend-following strategy evaluated over that window is evaluated over the period in which trend following is known to have worked (Hurst et al., 2017); CTA 1.0’s own commentary observes that the system “lost its mojo in 2009”, and none of the statistics here are computed sub-period.

The cost model of Table 2 is linear in traded notional with a single rate across 49 instruments, which is a crude proxy for a market impact that is neither linear nor uniform. It is used to rank, not to price. The break-even costs in Table 3 inherit that crudeness.

The assets are anonymised, so nothing here can be conditioned on asset class, liquidity or contract size, and the cross-sectional constructions of (6) and (9) cannot be checked for the concentrations they might be taking. Asset counts vary over the panel — the earliest rows carry a fraction of the 49 — and (9) handles that by restricting to live assets each day, so its effective universe grows through the sample.

No expected returns are estimated anywhere, no constraints on individual positions or groups are imposed, and no costs enter any objective. Adding a cost term to (9), which is where Table 2 suggests the next improvement lies, is exactly the regularisation the last notebook lists as future work and does not implement.

6 Conclusion

The ten-line CTA is a good pedagogical object because each of its flaws maps to one identifiable repair, and the repairs are separable enough to be measured one at a time. Measured that way, on this panel: the sequence’s Sharpe ratio gain is real and large, and it is not evenly distributed across the four steps; the risk properties improve monotonically where the Sharpe ratio does not, with excess kurtosis falling from 30.5 to 1.4; and the third step, the one whose Sharpe ratio contribution is zero, is the one that makes the sequence affordable, cutting turnover by two thirds and dominating the step it replaces at any positive trading cost.

The claim the notebooks make about parameter tuning holds and holds strongly. The best of 852 crossover pairs never reaches the untuned default of the system two steps later, and the parameter surfaces of the five systems are rank-correlated above 0.92 — the refinements lift the surface rather than reshaping it. Set against the selection bias that any best-of- K number carries, the case for spending effort on structure rather than search is about as clean as a single panel can make it.

What the sequence does not show, and what Table 3 makes uncomfortable, is that more machinery means more trading. CTA 5.0 turns over 147 times its AUM a year against CTA 1.0's 4, and at a one-way cost above about 12 basis points the ten lines win. The next refinement in this sequence should not be a better estimate of the correlation matrix. It should be a cost term in (9).

A Reproducibility

Every number above was produced from the repository at the commit that carries this paper, with the price panel in `book/marimo/notebooks/public/Prices_hashed.csv` and the project's locked environment. The five portfolios are built by the driver's own builders, so the strategy code is the notebooks':

```
import sys; sys.path.insert(0, "book/marimo/notebooks")
import numpy as np, optimize as O

BUILD = {1: lambda: O.build_exp1(fast=32, slow=96),
         2: lambda: O.build_exp2(fast=32, slow=96, volatility=32),
         3: lambda: O.build_exp3(fast=32, slow=96, vola=32, clip=4.2),
         4: lambda: O.build_exp4(fast=32, slow=96, vola=32, clip=4.2),
         5: lambda: O.build_exp5(vola=32, clip=4.2, corr=200, shrinkage=0.5)}

for k, build in BUILD.items():
    p = build(); s = p.stats
    ppy = s.periods_per_year
    to = p.turnover["turnover"].to_numpy()
    print(k, float(s.sharpe()["returns"]), float(s.volatility()["returns"]),
          float(s.max_drawdown()["returns"]), float(s.kurtosis()["returns"]),
          float(s.skew()["returns"]), np.nansum(to) / (len(to) / ppy))
```

Table 2 charges the cost through the same accounting, and Table 3 bisects the difference of two such curves on a 0.01 bps grid over [0, 40]:

```
r = p.returns["returns"].to_numpy(); to = p.turnover["turnover"].to_numpy()
def net_sharpe(bps):
    x = r - to * bps / 1e4
    return x.mean() / x.std(ddof=1) * np.sqrt(ppy)
```

Table 4 sweeps the grid

```
[(f, s) for f in range(4, 100, 4) for s in range(f + 4, 196, 4)]
```

through the same builders with the remaining parameters at their defaults, and Table 5 is `scipy.stats.spearmanr` on the resulting value vectors. Table 6 is the repository's driver verbatim:

```
python book/marimo/notebooks/optimize.py --experiment all --seed 42
```

The searches are seeded, so that table reproduces exactly; the grid and the cost curves are deterministic. The pinned Sharpe ratios in `tests/expected_sharpe.py` agree with column two of Table 1 to all printed digits, which is the check that the notebooks and the driver have not drifted apart.

References

- Akshay Agrawal et al. marimo: A reactive Python notebook. <https://marimo.io>, 2024.
- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.
- David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5): 94–107, 2014.
- Robert Engle. Dynamic conditional correlation: A simple class of multivariate generalized autoregressive conditional heteroskedasticity models. *Journal of Business & Economic Statistics*, 20(3):339–350, 2002.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
- Campbell R. Harvey and Yan Liu. Backtesting. *The Journal of Portfolio Management*, 42(1): 13–28, 2015.
- Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.
- Brian Hurst, Yao Hua Ooi, and Lasse Heje Pedersen. A century of evidence on trend-following investing. *The Journal of Portfolio Management*, 44(1):15–29, 2017.
- Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.
- Andrew W. Lo. The statistics of sharpe ratios. *Financial Analysts Journal*, 58(4):36–52, 2002.
- Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- Richard O. Michaud. The Markowitz optimization enigma: Is ‘optimized’ optimal? *Financial Analysts Journal*, 45(1):31–42, 1989.
- Tobias J. Moskowitz, Yao Hua Ooi, and Lasse Heje Pedersen. Time series momentum. *Journal of Financial Economics*, 104(2):228–250, 2012.
- Thomas Schmelzer and Raphael Hauser. Seven sins in portfolio optimization. *arXiv preprint arXiv:1310.3396 [q-fin.PM]*, 2013.
- Ritchie Vink et al. Polars: Blazingly fast DataFrames in Rust and Python. <https://pola.rs>, 2024.